

Deep Learning Practical 8

Build a small GPT

1 Next-character prediction

We will train a small GPT to continue passages from Shakespeare by predicting one character at a time. To understand how it uses the text already written, we will begin with independent character predictions, add a way to combine context, then learn how to weight that context using attention.

During training, the text gives us both the context and the correct next character. For example, the word **stars** gives four prediction tasks: after seeing **s**, predict **t**; after seeing **st**, predict **a**; and so on. We can store all four tasks in two aligned arrays, as shown below.

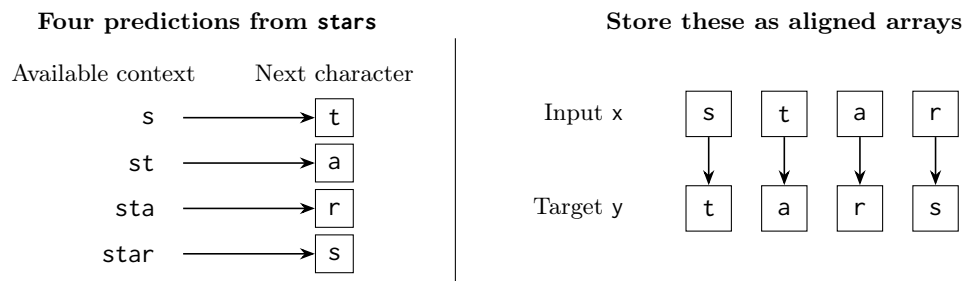


Figure 1: Aligned inputs and next-character targets for **stars**. To predict the target **r** in column 3, the model may use the input prefix **sta**.

Training on these aligned targets allows many predictions in one forward pass, provided each position can use only the input prefix ending there. Otherwise, later inputs would reveal the answer it is being trained to predict.

Download [dl-practical8.py](#) and work in the same directory. The following command loads its definitions so that we can build and inspect the model in stages:

```
%run -n dl-practical8.py
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
torch.set_num_threads(2)
torch.manual_seed(3801)
train_tokens = text_data().to(device)
val_tokens = text_data("validation")
print(bytes(train_tokens[:500].tolist()).decode("ascii"))
```

Because every character in this corpus occupies one ASCII byte, its integer value can index a learned embedding: a vector that represents the character within the model. We allow all 256 byte values and split the text into contiguous training, validation and test portions before sampling windows.

For longer passages we use the same alignment, with 256 input characters per window and the following character as each position's target:

```
x, y = text_batch(train_tokens, batch=2, context=256)
print(x.shape, y.shape) # both [2, 256]
print(bytes(x[0, :80].tolist()).decode("ascii"))
print(bytes(y[0, :80].tolist()).decode("ascii"))
assert torch.equal(x[:, 1:], y[:, :-1])
```

Exercise

For the word `sunny`, draw the four context–target pairs in the style of Figure 1, then write the input and target arrays. Using two overlapping windows from a longer passage, explain why randomly assigning windows to training and validation could give an unreliable validation score.

2 Context dependence

As a first attempt, suppose we look up an embedding for each input character and apply a linear layer to predict the next one. The linear layer produces 256 logits at each position, which softmax can turn into probabilities over the next byte. Although we pass in a whole sequence, these operations process its positions independently:

```
independent = nn.Sequential(nn.Embedding(256, 32), nn.Linear(32, 256))
phrases = torch.tensor([list(b"sta"), list(b"sea")])
logits = independent(phrases)
print(logits.shape)                # [2, 3, 256]
print(torch.equal(logits[0, -1], logits[1, -1])) # True
```

Both inputs end in `a`, so the final position retrieves the same embedding and produces the same probabilities, regardless of whether the preceding letters are `st` or `se`. To distinguish these contexts, we need an operation that combines information from several positions before predicting the next character.

Exercise

Draw the path from the final input character to its output, then consider increasing the embedding width or adding another linear layer independently at every position. Would either change allow an earlier character to influence that output? Check your reasoning using two other inputs that end in the same character.

3 Context averaging

We can begin combining positions by averaging their embeddings. At the third position, for example, the average should include the first three characters, while at the first position only one character is available. This gives a lower-triangular matrix of weights, whose rows describe the permitted combinations.

To inspect this operation by itself, use six random vectors as stand-ins for character embeddings. Multiplying them by the weight matrix produces one average per position:

```
length = 6
allowed = torch.ones(length, length, dtype=torch.bool).tril()
average = allowed.float()
average = average / average.sum(dim=-1, keepdim=True)
h = torch.randn(length, 4)
mixed = average @ h
print(average)
torch.testing.assert_close(mixed[2], h[:3].mean(dim=0))
plt.imshow(average, cmap="Greys", vmin=0, vmax=1)
plt.xlabel("Input position"); plt.ylabel("Prediction position")
plt.colorbar(); plt.show()
```

Keeping future positions at zero enforces the restriction from Section 1, which is called a *causal mask*. During training we already know the whole passage, but the mask ensures that each prediction has access only to the characters that would be available during generation.

Averaging therefore introduces context, although it treats every available character equally and loses their order: the final average for `star` would be the same as for `rats`. To represent more useful distinctions, we will give the model position information and let it learn how to weight the characters when combining them.

Exercise

Write the final row of `average` by hand, then replace the last two rows of `h` and check whether the first four rows of `mixed` change. What information would become available to the first prediction if every weight were $1/6$?

4 Causal self-attention

With attention, processing the third character of **stars** involves computing weights from the available representations of **s**, **t** and **a**, then using those weights to combine them.

To make the weights depend on order, add a learned position vector to each character embedding. Learned linear maps then produce a *query*, *key* and *value* for each position: comparing a query with the available keys determines how their value vectors are combined, as shown in Figure 2.

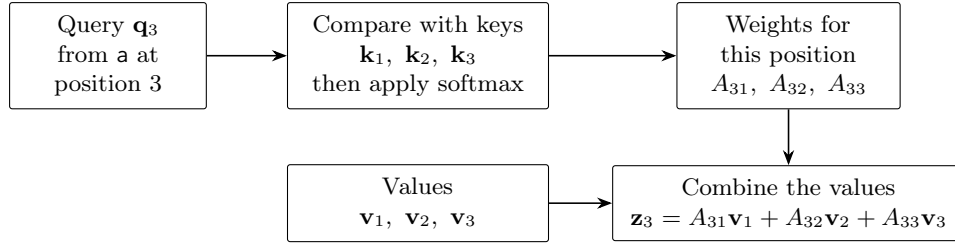


Figure 2: At position 3, comparing the query with the available keys determines how their values are combined; later positions are excluded.

If $\mathbf{H}$ contains the input vectors, the three projections are

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{H}\mathbf{W}_V.$$

We compute query–key similarities using dot products, then apply the mask and a row-wise softmax to obtain weights that sum to one:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{M}\right), \quad \mathbf{Z} = \mathbf{A}\mathbf{V}.$$

Here d is the query/key width, and M_{ij} is zero for $j \leq i$ and $-\infty$ for future positions. Consequently, A_{ij} is the weight that position i assigns to the value at position j . The following code performs this calculation with randomly initialised projections; training will learn the weights of those projections.

```

ids = torch.tensor(list(b"stars "))
token_embedding = nn.Embedding(256, 8)
position_embedding = nn.Embedding(256, 8)
h = token_embedding(ids) + position_embedding(torch.arange(len(ids)))
q, k, v = nn.Linear(8, 24, bias=False)(h).chunk(3, dim=-1)
scores = q @ k.T / math.sqrt(8)
weights = scores.masked_fill(~allowed, float("-inf")).softmax(-1)
z = weights @ v
print(z.shape) # [6, 8]
torch.testing.assert_close(weights.sum(-1), torch.ones(6))
plt.imshow(weights.detach(), cmap="Greys", vmin=0, vmax=1)
plt.xlabel("Key position"); plt.ylabel("Query position")
plt.colorbar(); plt.show()
  
```

Exercise

- Use the first row of the weight matrix to explain why the first output equals the first value vector. Why is it valid to include the diagonal when predicting the next character?
- Replace $-\infty$ with zero and inspect the weights assigned to future positions. Explain the result using the softmax formula.
- Multiply the scores by 10 before softmax and describe how the weights change. What does this suggest about the purpose of dividing by $\sqrt{d}$?

Multi-head attention

One weighted combination may discard information that would be useful for prediction, so multi-head attention learns several combinations in parallel. Each head has its own queries, keys and values; we concatenate its

output with those of the other heads and apply a learned projection. In our width-64 model, two heads each produce 32 features per position.

PyTorch combines the scaling, masking, softmax and weighted sum in `scaled_dot_product_attention`, allowing us to implement the same calculation compactly:

```
class Attention(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        self.heads = heads
        self.qkv = nn.Linear(width, 3*width, bias=False)
        self.output = nn.Linear(width, width)

    def forward(self, x):
        batch, length, width = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q, k, v = [a.reshape(batch, length, self.heads, width//self.heads)
                    .transpose(1, 2) for a in (q, k, v)]
        h = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        return self.output(h.transpose(1, 2).reshape(batch, length, width))
```

For a batch of 2 sequences of length 256, the input shape is (2, 256, 64). Splitting the feature dimension into two heads gives queries, keys and values of shape (2, 2, 256, 32), with axes for batch, head, position and features; concatenating the outputs restores the original shape.

Exercise

Compare the manual calculation with the PyTorch function for one head:

```
fast = F.scaled_dot_product_attention(
    q[None, None], k[None, None], v[None, None], is_causal=True)
torch.testing.assert_close(fast[0, 0], z, atol=1e-6, rtol=1e-5)
```

By considering the shape of the query-key score matrix, explain how doubling the sequence length affects the number of comparisons and whether the learned projection matrices need to grow.

5 Transformer architecture

Now that attention can combine context, we add a small feed-forward network to transform the resulting features at each position. Residual connections carry each operation's input forward, while layer normalisation controls the feature scale before the operation. Together, these components form a pre-normalised Transformer block:

```
class Block(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        self.norm1, self.norm2 = nn.LayerNorm(width), nn.LayerNorm(width)
        self.attention = Attention(width, heads)
        self.mlp = nn.Sequential(nn.Linear(width, 4*width), nn.GELU(),
                                  nn.Linear(4*width, width))

    def forward(self, x):
        x = x + self.attention(self.norm1(x))
        return x + self.mlp(self.norm2(x))
```

Adding input embeddings and a linear output layer turns the block into a language model. Its next-character scores at every position let it learn the aligned targets from Section 1; during generation, we use the final position as shown in Figure 3. Further blocks can be stacked before the output layer.

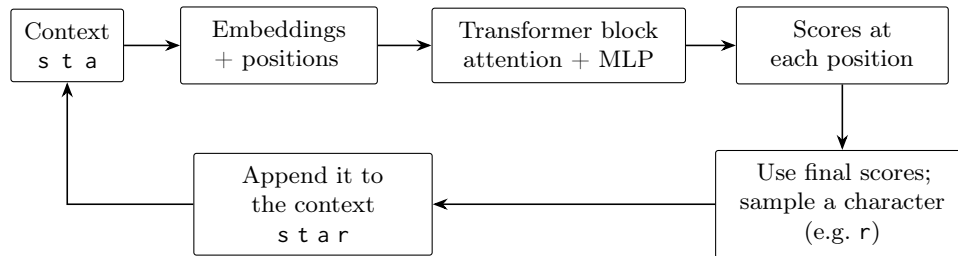


Figure 3: Generation extends the context with each sampled character, while training uses the known text to learn from all positions together.

The starter below uses one block, with learned token and position embeddings at its input:

```

class Model(nn.Module):
    def __init__(self, width=64, heads=2, layers=1):
        super().__init__()
        self.token = nn.Embedding(256, width)
        self.position = nn.Embedding(256, width)
        self.blocks = nn.Sequential(*(Block(width, heads) for _ in range(layers)))
        self.norm = nn.LayerNorm(width)
        self.head = nn.Linear(width, 256)

    def forward(self, tokens):
        length = tokens.size(1)
        position = torch.arange(length, device=tokens.device)
        h = self.token(tokens) + self.position(position)
        return self.head(self.norm(self.blocks(h)))

```

```

model = Model().to(device)
print(model(x).shape)                # [2, 256, 256]
print(sum(p.numel() for p in model.parameters())) # 99328

```

The two output dimensions of size 256 have different meanings: for each of the 256 context positions, the model produces 256 scores over possible next bytes.

Exercise

Check that changing future inputs leaves earlier predictions unchanged:

```

model.eval()
changed = x.clone()
changed[:, 100:] = (changed[:, 100:] + 17) % 256
with torch.no_grad():
    torch.testing.assert_close(model(x)[: , :100], model(changed)[: , :100])

```

Trace how an earlier character can influence a later prediction through attention and the feed-forward network. Which operations combine positions, and which transform the features separately at each position?

6 Training and validation

For training, we compare each position’s prediction with the character that actually follows it in the text. Averaging the negative log-probability of these targets gives the loss

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[-\frac{1}{T} \sum_{i=1}^T \log p_{\theta}(x_{i+1} \mid x_1, \dots, x_i) \right],$$

where $\mathbf{x} = (x_1, \dots, x_{T+1})$ is a window drawn from the training text. Cross entropy computes this loss directly from the logits, using natural logarithms; because each target is one byte, we report the result in nats per byte.

The supplied training function samples batches of windows and applies AdamW updates to reduce this loss. Run it with the starter settings, then evaluate on the separate validation text to see how well the learned predictions carry over to unseen passages:

```

torch.manual_seed(3801)
model = Model().to(device)
history = train(model, device)          # 2000 updates, batch size 32
steps, losses = zip(*history)
plt.plot(steps, losses, color="black")
plt.xlabel("Update"); plt.ylabel("Training NLL (nats/byte)")
plt.show()
score = language_metrics(model, val_tokens, device=device)
print(score["nll_nats_per_byte"])

```

The starter reaches approximately 1.783 nats per byte on validation, where lower values indicate better predictions. For a consistent comparison between models, the evaluator resets context at each 256-byte window boundary and counts every target once, including those in the final short window.

Exercise

- Locate the forward pass, loss, gradient calculation and parameter update in `train`. Explain why the logits and targets are flattened together, and why cross entropy receives logits directly.
- Calculate the NLL of a model that assigns probability $1/256$ to every byte, then compare it with the trained model's score.
- Try more updates and record both training loss and validation NLL. Use their behaviour to assess whether the additional training improves generalisation.

7 Text generation

With the trained model, we can carry out the loop in Figure 3: use the final position's scores to sample a character, append it to the context, and run the model again. Dividing the logits by a temperature below one concentrates probability on the more likely continuations, while keeping only the last 256 input characters allows the generated passage to grow beyond the training context length.

```

@torch.no_grad()
def generate(model, tokens, length=512, temperature=.8, generator=None):
    model.eval()
    for _ in range(length-tokens.size(1)):
        logits = model(tokens[:, -256:][:, -1] / temperature
        next_token = torch.multinomial(logits.softmax(-1), 1, generator=generator)
        tokens = torch.cat([tokens, next_token], dim=1)
    return tokens

```

```

prompt = torch.tensor([list(b"ROMEO:\n")], device=device)
torch.manual_seed(921)
sample = generate(model, prompt, length=512, temperature=.8)
print(bytes(sample[0].tolist()).decode("utf-8", errors="replace"))

```

The passages below were generated with the starter settings and seed 3801, alongside a score measured on the complete validation split.

gpt: 99,328 parameters, 2000 updates, batch 32; NLL/byte 1.783.

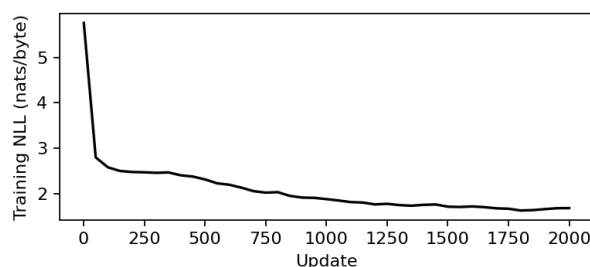


Figure 4: Training loss from this run.

These are the first two generated passages, with their original line breaks; because sampling stops at 512 bytes, the final sentence may be incomplete.

ROMEO:

the of dit shall bodgine thusbare far,
Less thy beggare enter.

BRUTUS:

I will of York:
Thou heave marcian my makes of the sover circh, he bass'd,
Go the stard's in matter.

GLOUCESTEL:

That of thy be too thou we means; like that,
What's angers:
God in
To keep thy make duke an the word, this worl!
I meet not fried that all be the that wam come upon to use sterpide to much gaze any sword on the go
By the call out lord one.

COMINIUS:

Then and to the with a sirst way the with not and Here with his it

JULIET:

The is grough child deart of the to the to disching;
From thy deping be me!

CORIOLANUS:

How pritione the noble thed nevold bemia: deads with here the news
Is and by enoish noble, meanusing queen?
Merce's leands mother; a pear.

HENRY VI:

The goody sthy gue befedeed thou arewer wall here fore with the grave me; is that stoch the murship!

LADY GLIUS:

When bleed stand whom.

QUEEN MIONE:

Who, to that he good's with the seem thath.

JULIET:

He Take the from me bare fore-morrshes his of desee in bef'

Exercise

Compare temperatures 0.5, 0.8 and 1.2 with the same model and prompt, looking for changes in spelling, repeated phrases and speaker transitions. Explain why these samples can change even though the validation NLL remains the same.

8 Improve the model

Experiment with the number of blocks, their width and the training settings, recording the parameter count and validation NLL alongside generated passages. If a larger model overfits, investigate dropout, weight decay or a decaying learning rate; sharing input and output embedding weights can also reduce the parameter count.

To examine how a trained model uses context, repeat the attention calculation from Section 4 with its learned projections in `model.blocks[0].attention`. First add the token and position embeddings and apply `norm1`, matching the order in the block, then plot each head's weights for a short phrase.

Exercise

Compare the two heads on a phrase such as `to be or not to be`, then change part of the phrase and inspect the weights again. Relate the differences to the weighted sum in Figure 2, and explain which other operations affect how an earlier character influences the output.

Further reading. D2L, Chapter 11: attention and Transformers, https://d2l.ai/chapter_attention-mechanisms-and-transformers/. Compare the model code with <https://github.com/karpathy/nanoGPT>.

9 Submission

Download `d1-practical8.py` , set `STUDENT_ID` and edit the model and training settings. Run the complete file or paste the whole program into one notebook cell. It trains, scores and creates the ZIP below; upload this to the corresponding Ultra item.

```
lab-08-studentID.zip
lab-08-code.py
lab-08-results.json
samples/
  batch-01.txt
  batch-02.txt
  ...
  batch-10.txt
```

Details and constraints

- Generate ten text files, each containing eight passages of 512 bytes with their original line breaks. Keep the fixed prompts, sample headings and separators.
- Use at most 1,000,000 parameters and buffer elements in total, 10,000 optimiser updates and a batch size of 64.
- Train from fresh weights on the supplied Shakespeare training split, with a context of 256 bytes.
- Use no extra data, pretrained models, distillation teacher, gradient accumulation or extra training batches per update.
- Keep the loss, held-out evaluation and sampling code fixed. Samples use temperature 0.8. The score is next-byte NLL in nats per byte, calculated independently of sampling temperature.
- Put a brief abstract at the top of the Python file, with references and AI acknowledgements in comments. The program writes the JSON and sample files; submit these unchanged.

Marking

- Marks reflect the quality and simplicity of your solution and its measured performance within the constraints. Aim for next-byte NLL below 1.783 nats per byte.
- A pass requires a small improvement over the baseline. A 2:1 requires significant improvements using established techniques discussed in the course. A high first requires substantial improvements through well-implemented state-of-the-art methods, for example by integrating methods from recent papers, producing high-quality samples and scores within the budget.
- The exact marking function is withheld to avoid gamification.

Solutions for this assignment cannot later be publicised without request. Requests will be considered on a case-by-case basis after marking.